

Desafio Cérebro da Lu

Olá! Seja muito bem-vindo(a) ao nosso desafio técnico. Este teste foi desenhado para avaliar suas habilidades na interseção entre o desenvolvimento de software tradicional e a engenharia de inteligência artificial/LLMs.

O tempo estimado para a realização deste teste é de **4 a 6 horas**, mas você terá um prazo de **uma semana** para nos entregar a solução a partir do recebimento deste documento.

O Objetivo

Você deve construir uma **API REST conversacional** utilizando FastAPI e integrar com o framework de agentes Google ADK.

Essa API intermediará a conversa entre um cliente e um **Agente de Vendas Inteligente**. O agente deve ser capaz de gerenciar o carrinho de compras do usuário, simular a finalização de uma compra via PIX e consultar o status de pedidos anteriores.

Requisitos Funcionais

O agente deve ser capaz de interpretar os comandos textuais do usuário e executar as seguintes ações por meio de **Tools (Function Calling)**:

- Gerenciamento de Carrinho (Persistente entre sessões):**
 - **Adicionar Produto:** Incluir um item e sua respectiva quantidade no carrinho do usuário.
 - **Remover Produto:** Retirar um item específico do carrinho.
 - **Limpar Carrinho:** Esvaziar completamente o carrinho do usuário.
- Checkout Simulado (Apenas PIX):**
 - O usuário pode solicitar o fechamento do pedido.
 - O sistema deve esvaziar o carrinho, simular a criação de uma chave PIX "copia e cola" e gerar um **ID de Pedido** único.
- Consulta de Pedidos:**
 - O usuário deve conseguir perguntar o status de um ID de pedido.
 - O resultado da consulta deve ser aleatório entre: **Processando** ou **Enviado**. Não precisa persistir o resultado.
 - Se o ID de pedido não existir na base de dados fictícia, o agente deve informar que o pedido não foi encontrado.

4. Estrutura agêntica

- Você pode definir qual estrutura utilizar, um agente ou multi-agente.

5. Resposta da API

- A resposta final da sua API para o usuário deve ser **estritamente o texto gerado pelo agente**. Nenhum outro evento interno do ADK ou logs de execução devem ser expostos no payload de resposta.

Contrato da API

A comunicação será feita através de um único endpoint de chat. O estado do carrinho e o histórico de conversa devem ser mantidos usando o `session_id`.

- **Endpoint:** `POST /api/v1/chat`
- **Content Type:** `application/json`
- **Descrição:** Envia uma mensagem do usuário para o agente de vendas. O agente processa a intenção, manipula o carrinho se necessário e retorna a resposta textual.

Request Body (JSON):

```
{
  "session_id": "user-session-12345",
  "message": "Quero adicionar 2 tênis de corrida ao meu carrinho."
}
```

Response Body (JSON) - Sucesso (Status 200):

```
{
  "session_id": "user-session-12345",
  "response": "Perfeito! Já adicionei 2 tênis de corrida ao seu carrinho. Deseja adicionar mais alguma coisa ou quer fechar o pedido?"
}
```



Requisitos Técnicos e Premissas

- **Linguagem:** Python
- **Banco de Dados / Persistência:** O dado pode permanecer em memória para efeitos do teste
- **Modelo de Linguagem (LLM):** Você pode utilizar qualquer provedor (OpenAI, Anthropic, Google AI Studio) ou rodar um modelo localmente (via Ollama). Dica: Modelos como o Gemini Flash ou Llama 3 via Groq possuem camadas gratuitas.
- **Variáveis de Ambiente:** Não envie chaves de API (chaves privadas) no código. Utilize um arquivo `.env` (envie apenas um `.env.example`).



Instruções de Entrega

1. Disponibilize o código em um repositório **privado** no GitHub e dê acesso de leitura para os usuários: `[leosilvalabs]`
2. O repositório deve conter um arquivo `README.md` com:
 - a. Instruções claras de como rodar a aplicação localmente (preferencialmente usando `docker-compose`).
 - b. Explicação sucinta da arquitetura escolhida e das ferramentas utilizadas.

Boa sorte! Estamos ansiosos para ver a sua solução. 🚀